

LAB 4. ENVIRONMENTAL SOUND CLASSIFICATION

URBANSOUND8K WITH CRNN

ThS. Lê Thị Thùy Trang

2026-04-27

1 Giới thiệu bài thực hành

1.1 Mục tiêu bài thực hành

Sau Lab 3, sinh viên đã biết cách đưa âm thanh về dạng đặc trưng theo thời gian và dùng **1D-CNN** để học các mẫu cục bộ. Sang bài lab này, chúng ta nâng cấp mô hình theo một hướng rất tự nhiên: dùng **CNN** để trích đặc trưng từ spectrogram, sau đó dùng **RNN** để học diễn biến của âm thanh theo thời gian. Mô hình kiểu này thường được gọi là **CRNN**: Convolutional Recurrent Neural Network.

Mục tiêu chính của buổi học không phải là chạy một mô hình thật to cho oách, mà là hiểu vì sao audio classification không chỉ là phát hiện một vài pattern ngắn, mà còn cần quan sát sự thay đổi của tín hiệu theo thời gian. Trong starter kit, cấu hình chính được chọn là **log-mel spectrogram + CRNN-GRU** để lab chạy ổn trên laptop cá nhân. Cấu hình mở rộng **BiLSTM-CRNN** cũng đã được chuẩn bị để sinh viên khá thử nghiệm thêm.

- Hiểu vì sao CRNN phù hợp với dữ liệu âm thanh dạng chuỗi thời gian–tần số;
- Biết cách biểu diễn audio bằng log-mel spectrogram;
- Xây dựng và huấn luyện mô hình CRNN cho bài toán phân loại âm thanh môi trường;
- Hiểu vai trò khác nhau của CNN block và RNN block trong một mô hình CRNN;
- Dùng **Weights & Biases** để log quá trình huấn luyện;
- Đọc learning curves và phát hiện overfitting/underfitting;
- Phân tích confusion matrix để biết mô hình hay nhầm lớp nào;
- So sánh kết quả của CRNN ở Lab 4 với 1D-CNN ở Lab 3;
- Rút ra kết luận dựa trên số liệu thay vì cảm giác.

1.2 Kết quả đầu ra mong đợi của bài lab

Sau bài lab này, sinh viên cần có:

1. Một repo chạy được với bài toán phân loại âm thanh bằng CRNN;
2. Một pipeline tiền xử lý audio chạy được trên tập UrbanSound8K;
3. Một mô hình log-mel + CRNN-GRU chạy ổn và có log trên W&B;
4. Ít nhất 1 run chính cho baseline log-mel + CRNN;
5. Learning curves cho train và validation;
6. Confusion matrix cho kết quả test;

7. Một bảng tổng hợp metrics và thời gian train/epoch;
8. Một phần so sánh ngắn với kết quả 1D-CNN ở Lab 3;
9. Phân tích ngắn: khi nào CRNN có thể tốt hơn 1D-CNN và khi nào chưa chắc tốt hơn.

1.3 Những thứ bắt buộc phải nộp

Mỗi sinh viên cần nộp tối thiểu:

1. Repo code chạy được.
2. README hướng dẫn chạy.
3. Link hoặc ảnh chụp W&B dashboard.
4. Kết quả của mô hình baseline `log-mel + CRNN-GRU`.
5. File `curves.png`.
6. File `confusion_matrix.png`.
7. File `metrics.json` hoặc bảng tổng hợp metrics.
8. Kết quả test của best model.
9. Bảng so sánh tối thiểu giữa 1D-CNN ở Lab 3 và CRNN ở Lab 4.
10. Phân tích ngắn: lớp nào dễ bị nhầm, vì sao có thể bị nhầm.
11. Bài mở rộng nếu có: so sánh thêm BiLSTM-CRNN hoặc biến thể hyperparameter khác.

1.4 Bối cảnh bài toán

Chúng ta tiếp tục đóng vai nhóm kỹ sư AI xây dựng hệ thống nhận diện âm thanh môi trường trong đô thị. Mỗi file đầu vào là một đoạn âm thanh ngắn, ví dụ tiếng còi xe, tiếng chó sủa, tiếng khoan, tiếng điều hòa, tiếng trẻ em chơi hoặc tiếng nhạc đường phố. Mô hình cần dự đoán đoạn âm thanh đó thuộc loại nào.

Bộ dữ liệu sử dụng vẫn là **UrbanSound8K**, sinh viên có thể truy cập đường dẫn [này](#) để lấy dữ liệu. Tập dữ liệu gồm 10 lớp âm thanh môi trường:

- `air_conditioner`
- `car_horn`
- `children_playing`
- `dog_bark`
- `drilling`
- `engine_idling`
- `gun_shot`
- `jackhammer`
- `siren`
- `street_music`

Nếu ở Lab 3 chúng ta dùng 1D-CNN để học pattern cục bộ trên chuỗi đặc trưng audio, thì ở Lab 4 chúng ta dùng CRNN để học theo hai bước rõ ràng hơn:

audio -> log-mel spectrogram -> CNN blocks -> RNN/GRU/LSTM -> classifier

Nói dễ hiểu hơn, CNN giúp mô hình nhìn ra các mẫu cục bộ trên bản đồ thời gian–tần số, còn RNN giúp mô hình quan sát diễn biến của các mẫu đó theo thời gian.

1.5 Cấu trúc repo starter kit

Repo starter kit của bài lab này có tên đề xuất:

```
csc4005_lab4_urbansound8k_crnn_starter
```

Cấu trúc thư mục chính như sau:

```
csc4005_lab4_urbansound8k_crnn_starter/  
  README.md  
  REPORT_TEMPLATE.md  
  requirements.txt  
  configs/  
    baseline_logmel_crnn.json  
    fast_debug.json  
    extension_bilstm_crnn.json  
  docs/  
    WANDB_GUIDE.md  
    RUBRIC.md  
    LAB_GUIDE_LAB4.md  
  notebooks/  
    lab4_demo.ipynb  
  outputs/  
  src/  
    dataset.py  
    model.py  
    train.py  
    utils.py  
  ci/  
    check_structure.py  
    smoke_train.py
```

Trong đó:

- `configs/fast_debug.json`: cấu hình chạy nhanh để kiểm tra môi trường;
- `configs/baseline_logmel_crnn.json`: cấu hình chính của bài lab;
- `configs/extension_bilstm_crnn.json`: cấu hình mở rộng dùng BiLSTM;
- `src/dataset.py`: đọc dữ liệu, chia fold, trích xuất log-mel/MFCC;
- `src/model.py`: định nghĩa CRNN;
- `src/train.py`: script huấn luyện, đánh giá, lưu output và log W&B;
- `src/utils.py`: các hàm phụ trợ như lưu metrics, vẽ curves, confusion matrix;
- `outputs/`: nơi lưu kết quả sau khi train.

Điểm quan trọng là repo này **không chứa dữ liệu UrbanSound8K**. Dữ liệu phải được để ngoài repo và truyền vào bằng tham số `--data_dir`. Không push thư mục `data/`, file `.zip` lớn, cache đặc trưng hoặc model checkpoint lớn lên GitHub.

2 Chuẩn bị môi trường thực hành

2.1 Điều kiện cần

Trước khi bắt đầu, sinh viên cần có:

- Python 3.10 hoặc 3.11;
- Git;
- Tài khoản GitHub;
- Tài khoản `Weights & Biases`;
- Bộ dữ liệu `UrbanSound8K`;
- Laptop cá nhân hoặc môi trường cloud có thể cài `PyTorch` và `librosa`.

Nếu máy không có GPU cũng vẫn làm được bài này, nhưng thời gian train sẽ lâu hơn. Cấu hình `fast_debug.json` được chuẩn bị để kiểm tra nhanh môi trường trước khi chạy baseline đầy đủ.

2.2 Clone repo

Sau khi nhận được link GitHub Classroom hoặc link starter repo, sinh viên clone repo về máy:

```
git clone <link-repo-cua-sinh-vien>
cd csc4005_lab4_urbansound8k_crnn_starter
```

Nếu dùng GitHub Classroom, tên repo của mỗi sinh viên có thể khác nhau. Điều này không sao. Quan trọng là cấu trúc thư mục bên trong repo phải giữ đúng.

2.3 Khởi tạo environment

Với macOS hoặc Linux:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

Với Windows:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
```

2.4 Cài thư viện phục vụ bài lab

Cài các thư viện cần thiết:

```
pip install -r requirements.txt
```

File `requirements.txt` đã bao gồm các thư viện chính như `torch`, `numpy`, `pandas`, `matplotlib`, `scikit-learn`, `librosa`, `soundfile` và `wandb`.

Nếu gặp lỗi khi cài `librosa` hoặc `soundfile`, cần đọc kỹ thông báo lỗi. Trên một số máy, lỗi có thể đến từ môi trường Python cũ, thiếu compiler, hoặc package âm thanh chưa tương thích. Cách xử lý an toàn nhất là dùng Python 3.10/3.11 và tạo môi trường mới.

2.5 Chuẩn bị dữ liệu UrbanSound8K

Sau khi tải và giải nén, cấu trúc dữ liệu UrbanSound8K nên có dạng:

```
UrbanSound8K/
  metadata/
    UrbanSound8K.csv
  audio/
    fold1/
    fold2/
    fold3/
    ...
    fold10/
```

Ví dụ, nếu dữ liệu đặt ở thư mục `/Users/trang/datasets/UrbanSound8K`, thì khi chạy train sẽ truyền:

```
--data_dir /Users/trang/datasets/UrbanSound8K
```

Có thể dùng đường dẫn tới thư mục đã giải nén hoặc file `.zip`. Tuy nhiên, để tránh lỗi và tiết kiệm thời gian, nên giải nén sẵn dữ liệu trước khi chạy nhiều thí nghiệm.

Trong starter kit, cấu hình mặc định dùng chia fold như sau:

- train folds: 1,2,3,4,5,6,7,8
- validation fold: 9
- test fold: 10

Không nên trộn ngẫu nhiên toàn bộ file audio rồi chia train/validation/test nếu chưa hiểu rõ nguy cơ rò rỉ dữ liệu. UrbanSound8K đã cung cấp fold, vì vậy bài lab này dùng fold để kết quả dễ so sánh và rõ ràng hơn.

2.6 Nhắc lại thao tác với W&B

Đăng nhập W&B bằng lệnh:

```
wandb login
```

Sau đó copy API key từ tài khoản W&B và dán vào terminal. Nếu đang chạy trên máy cá nhân, thao tác này thường chỉ cần làm một lần.

Để kiểm tra nhanh đã cài được wandb chưa:

```
python -c "import wandb; print(wandb.__version__)"
```

3 Voila, hướng dẫn thực hành ở đây

3.1 Kiểm tra pipeline bằng fast debug

Trước khi train baseline đầy đủ, hãy chạy cấu hình debug nhanh. Mục tiêu của bước này không phải là đạt accuracy cao, mà là kiểm tra xem pipeline có chạy xuyên suốt hay không:

```
python -m src.train \
  --config configs/fast_debug.json \
  --data_dir /duong_dan/UrbanSound8K \
  --use_wandb
```

Nếu máy chạy Windows, có thể viết lệnh trên một dòng:

```
python -m src.train --config configs/fast_debug.json --data_dir C:\duong_dan\UrbanSound8K --use_wandb
```

Run debug dùng cấu hình nhẹ hơn:

- target_sr = 8000
- duration = 1.0
- n_mels = 32
- model_name = crnn_tiny
- epochs = 2

Nếu debug chạy được, sinh viên đã vượt qua phần dễ gây lỗi nhất: đọc audio, trích xuất đặc trưng, tạo batch tensor, đưa qua mô hình CRNN và log lên W&B.

Sau khi chạy xong, kiểm tra thư mục:

outputs/debug_logmel_crnn/

Trong đó nên có các file:

- best_model.pt
- history.csv
- curves.png
- confusion_matrix.png
- metrics.json

Nếu chưa có các file này, chưa nên chạy baseline dài. Hãy sửa lỗi debug trước.

3.2 Chạy baseline chính: log-mel + CRNN-GRU

Sau khi debug thành công, chạy baseline chính:

```
python -m src.train \  
--config configs/baseline_logmel_crnn.json \  
--data_dir /duong_dan/UrbanSound8K \  
--use_wandb
```

Cấu hình baseline chính trong file configs/baseline_logmel_crnn.json:

- feature_type = logmel
- model_name = crnn_small
- rnn_type = gru
- bidirectional = false
- target_sr = 16000
- duration = 4.0
- n_mels = 64
- n_fft = 1024
- hop_length = 512
- epochs = 25

- `batch_size = 32`
- `optimizer = adamw`
- `lr = 0.001`
- `weight_decay = 0.0001`
- `dropout = 0.3`
- `augment = true`

Với cấu hình này, mỗi audio được chuẩn hóa về 4 giây, chuyển thành log-mel spectrogram, rồi đưa vào mô hình CRNN. Input của mô hình có dạng gần như sau:

`[B, 1, n_mels, time_steps]`

Trong đó:

- `B`: batch size;
- `1`: một kênh spectrogram;
- `n_mels`: số dải mel;
- `time_steps`: số frame theo thời gian.

Với Lab 4, sinh viên không cần tự viết lại toàn bộ mô hình từ đầu trong buổi đầu tiên. Điều quan trọng là đọc được code, hiểu được dữ liệu đi qua từng khối như thế nào, biết chạy thí nghiệm và phân tích kết quả.

3.3 Vì sao dùng log-mel spectrogram cho CRNN?

Raw waveform là tín hiệu một chiều theo thời gian. Nó gần với dữ liệu gốc nhất, nhưng cũng khó học hơn vì mô hình phải tự phát hiện mọi thứ từ đầu. MFCC thì gọn và ổn định, rất tốt cho baseline nhẹ. Tuy nhiên, với CRNN, log-mel spectrogram thường trực quan hơn vì nó giữ được cấu trúc thời gian–tần số.

Có thể hình dung log-mel spectrogram như một bức ảnh đặc biệt:

- trục ngang là thời gian;
- trục dọc là tần số theo thang mel;
- giá trị tại mỗi điểm thể hiện năng lượng âm thanh.

Vì vậy, CNN có thể quét trên vùng cục bộ của bản đồ này để học những pattern như tiếng còi, tiếng khoan, tiếng chó sủa hoặc tiếng động cơ. Sau đó, RNN tiếp tục quan sát các pattern đó xuất hiện và thay đổi như thế nào theo thời gian.

3.4 CRNN đang học gì trong bài toán audio?

Một mô hình CRNN trong bài lab này có thể hiểu theo ba tầng ý tưởng:

1. **Feature extraction**: audio được biến thành log-mel spectrogram.
2. **Local pattern learning**: CNN học các mẫu cục bộ trên spectrogram.
3. **Temporal modeling**: GRU hoặc LSTM học chuỗi đặc trưng theo thời gian.

Trong code, phần CNN sẽ nhận input dạng:

`[B, 1, n_mels, time_steps]`

Sau vài block convolution và pooling, mô hình lấy trung bình theo trục tần số, rồi chuyển dữ liệu thành chuỗi:

`[B, T, C]`

Trong đó T là số bước thời gian còn lại, còn C là số đặc trưng tại mỗi bước thời gian. Chuỗi này được đưa vào GRU hoặc LSTM. Cuối cùng, trạng thái cuối cùng của RNN được dùng để phân loại 10 lớp âm thanh.

Nếu nói ngắn gọn:

CNN trả lời: trong đoạn âm thanh này có pattern gì?

RNN trả lời: các pattern đó diễn ra theo trình tự như thế nào?

Classifier trả lời: vậy đoạn âm thanh này thuộc lớp nào?

3.5 Nếu nhìn những biểu đồ như nhìn bức vách thì không sao hết, đọc hướng dẫn dưới đây là ổn áp ngay

Sau khi train, mở file:

`outputs/logmel_crnn_gru_baseline/curves.png`

hoặc xem trực tiếp trên dashboard W&B. Mục tiêu không phải là nhìn biểu đồ cho vui, mà là đọc được câu chuyện huấn luyện của mô hình.

3.5.1 Những đường quan trọng nhất

Có bốn đường cần quan sát:

- `train_loss`: loss trên tập train;
- `val_loss`: loss trên tập validation;
- `train_acc`: accuracy trên tập train;
- `val_acc`: accuracy trên tập validation.

Trong đó, `val_loss` và `val_acc` thường quan trọng hơn khi chọn mô hình, vì chúng cho biết mô hình có tổng quát tốt sang dữ liệu chưa thấy hay không.

3.5.2 Trường hợp tốt

Một run được xem là tương đối ổn nếu:

- `train_loss` giảm dần;
- `val_loss` cũng giảm hoặc dao động nhẹ quanh một mức thấp;
- `train_acc` tăng dần;
- `val_acc` tăng theo và không bị tụt mạnh;
- khoảng cách giữa train và validation không quá lớn.

Đây là dấu hiệu mô hình đang học được pattern có ích, chứ không chỉ học thuộc tập train.

3.5.3 Dấu hiệu overfitting

Overfitting thường xuất hiện khi:

- `train_loss` tiếp tục giảm;
- `train_acc` tiếp tục tăng;
- nhưng `val_loss` bắt đầu tăng;

- `val_acc` đứng yên hoặc giảm.

Với CRNN, overfitting có thể xảy ra khá nhanh nếu mô hình quá lớn hoặc dữ liệu không đủ đa dạng. Vì vậy baseline đã dùng dropout, weight_decay, augmentation và early stopping.

3.5.4 Dấu hiệu underfitting

Underfitting thường xuất hiện khi:

- cả `train_loss` và `val_loss` đều cao;
- cả `train_acc` và `val_acc` đều thấp;
- tăng epoch nhưng mô hình gần như không cải thiện.

Khi đó có thể mô hình quá nhỏ, learning rate chưa phù hợp, đặc trưng chưa tốt, hoặc quá trình train có lỗi. Không nên kết luận vội rằng “CRNN không tốt” chỉ sau một run chưa kiểm soát.

3.6 Đọc confusion matrix

Sau khi train xong, mở file:

`outputs/logmel_crnn_gru_baseline/confusion_matrix.png`

Confusion matrix cho biết mô hình dự đoán đúng/sai ở từng lớp. Đường chéo chính là số mẫu dự đoán đúng. Các ô ngoài đường chéo là các lỗi nhầm lẫn.

Khi đọc confusion matrix, cần trả lời các câu hỏi:

- Lớp nào có nhiều mẫu dự đoán đúng nhất?
- Lớp nào bị nhầm nhiều nhất?
- Hai lớp nào thường bị nhầm với nhau?
- Việc nhầm đó có hợp lý về mặt âm thanh không?
- So với Lab 3, CRNN có giảm nhầm lẫn ở lớp nào không?

Ví dụ, `drilling` và `jackhammer` có thể bị nhầm vì đều là âm thanh cơ khí mạnh. `children_playing` và `street_music` cũng có thể bị nhầm nếu background phức tạp. Phân tích như vậy sẽ tốt hơn nhiều so với chỉ ghi một dòng: “accuracy đạt X%”.

3.7 So sánh với Lab 3: 1D-CNN và CRNN

Đây là phần quan trọng nhất về mặt tư duy. Sinh viên cần lấy kết quả tốt nhất của Lab 3 và so sánh với baseline CRNN ở Lab 4. Có thể dùng bảng sau trong báo cáo:

Run	Feature	Model	Test acc	Avg epoch time	Nhận xét
Lab 3 baseline	MFCC	1D-CNN	...	...	...
Lab 4 baseline	log-mel	CRNN-GRU	...	...	...
Lab 4 extension	log-mel	BiLSTM-CRNN	...	...	...

Khi so sánh, không chỉ nhìn model nào accuracy cao hơn. Hãy nhìn thêm:

- mô hình nào train lâu hơn;
- mô hình nào ổn định hơn giữa train và validation;

- mô hình nào ít overfit hơn;
- mô hình nào giảm lỗi ở những lớp khó;
- mô hình nào phù hợp hơn nếu cần triển khai nhẹ.

Một mô hình phức tạp hơn không tự động là mô hình tốt hơn. Nếu CRNN chỉ tăng accuracy rất ít nhưng train lâu hơn nhiều, cần thảo luận xem việc tăng độ phức tạp có đáng hay không.

3.8 Thử bản mở rộng: BiLSTM-CRNN

Sau baseline, sinh viên muốn điểm cao có thể chạy thêm bản mở rộng:

```
python -m src.train \
  --config configs/extension_bilstm_crnn.json \
  --data_dir /duong_dan/UrbanSound8K \
  --use_wandb

  figs/extension_bilstm_crnn.json
--data_dir /duong_dan/UrbanSound8K
--use_wandb \end{verbatim}
```

Bản này dùng:

- `rnn_type = lstm`
- `bidirectional = true`
- `lr = 0.0007`
- `dropout = 0.35`

BiLSTM cho phép mô hình quan sát chuỗi theo cả hai chiều. Về mặt trực giác, mô hình có thêm thông tin ngữ cảnh trước và sau trong chuỗi đặc trưng. Tuy nhiên, nó cũng có nhiều tham số hơn và có thể train chậm hơn. Vì vậy cần so sánh bằng số liệu, không chỉ nói “BiLSTM nghe có vẻ xịn hơn”.

3.9 So sánh các run trên W&B

Sau khi có ít nhất hai run, vào dashboard W&B và so sánh:

- `logmel_crnn_gru_baseline`
- `logmel_crnn_bilstm_extension` nếu có
- run tốt nhất của Lab 3 nếu sinh viên đã log cùng tài khoản

Các tiêu chí nên xem:

- `best_val_acc`
- `test_acc`
- `val_loss`
- `avg_epoch_time_sec`
- `trainable_params`

Nếu một run có `train_acc` rất cao nhưng `val_acc` thấp, run đó không nên được chọn là mô hình tốt nhất. Nếu một run có test accuracy cao hơn một chút nhưng train lâu hơn rất nhiều, cũng cần cân nhắc khi viết kết luận.

3.10 Bước cuối cùng: so bó đũa, chọn cột cờ

3.10.1 Chọn best model

Trong starter kit, model tốt nhất theo validation loss được lưu tại:

```
outputs/<run_name>/best_model.pt
```

Ví dụ với baseline:

```
outputs/logmel_crnn_gru_baseline/best_model.pt
```

Không nên chọn model chỉ vì epoch cuối cùng có train accuracy cao. Cách chọn hợp lý hơn là dựa vào validation loss hoặc validation accuracy. Starter kit đã dùng early stopping để lưu model tốt nhất trong quá trình train.

Trong báo cáo, sinh viên cần ghi rõ:

- tên run được chọn;
- cấu hình chính của run đó;
- best validation accuracy;
- test accuracy;
- lý do chọn run này.

3.10.2 Đánh giá trên test set

Sau khi train xong, script tự động load `best_model.pt` và đánh giá trên test fold. Kết quả được lưu trong:

```
outputs/<run_name>/metrics.json
```

Trong file này, cần chú ý các trường:

- `best_val_acc`
- `test_acc`
- `test_loss`
- `avg_epoch_time_sec`
- `total_params`
- `trainable_params`
- `classification_report`
- `confusion_matrix`

Không nên báo cáo duy nhất một con số accuracy. Với bài toán nhiều lớp như UrbanSound8K, cần nhìn thêm từng lớp. Một mô hình có accuracy tổng thể ổn nhưng dự đoán rất kém ở một vài lớp vẫn cần được phân tích kỹ.

4 Khi nào CRNN tốt hơn 1D-CNN?

CRNN có thể tốt hơn 1D-CNN khi bài toán cần mô hình hóa diễn biến theo thời gian. Ví dụ một âm thanh không chỉ được nhận diện bởi một khoảnh khắc ngắn, mà bởi chuỗi pattern kéo dài: tiếng còi có thể tăng giảm cường độ, tiếng chó sủa có thể gồm nhiều nhịp, tiếng khoan có thể có cấu trúc lặp theo thời gian.

Tuy nhiên, CRNN không phải lúc nào cũng thắng. Một số lý do:

- dữ liệu không đủ lớn để mô hình phức tạp học tốt;

- cấu hình RNN chưa phù hợp;
- spectrogram hoặc preprocessing chưa tốt;
- mô hình overfit nhanh;
- 1D-CNN baseline đã đủ mạnh cho một số lớp âm thanh rõ ràng.

Vì vậy kết luận đúng không phải là “CRNN luôn tốt hơn 1D-CNN”, mà là: CRNN bổ sung khả năng học chuỗi thời gian, nhưng hiệu quả thực tế phải được kiểm chứng bằng thí nghiệm có kiểm soát.

5 Những lỗi rất hay gặp

5.1 Lỗi 1. Sai đường dẫn dữ liệu

Đây vẫn là lỗi phổ biến nhất. Hãy chắc chắn thư mục truyền vào `--data_dir` có chứa `metadata/UrbanSound8K.csv` và thư mục `audio/fold1`, `audio/fold2`, ...

5.2 Lỗi 2. Push dữ liệu hoặc model lớn lên GitHub

Không push `UrbanSound8K.zip`, thư mục `audio/`, cache đặc trưng hoặc file model lớn lên GitHub. Repo chỉ nên chứa code, config, tài liệu và kết quả nhỏ cần thiết.

5.3 Lỗi 3. Nhầm log-mel spectrogram với ảnh thông thường

Log-mel spectrogram có thể nhìn giống ảnh, nhưng ý nghĩa khác ảnh tự nhiên. Trục ngang là thời gian, trục dọc là tần số. Khi phân tích mô hình, cần nhớ dữ liệu vẫn là audio, không phải ảnh RGB thông thường.

5.4 Lỗi 4. Hiểu sai shape của input

Input của CRNN trong bài lab này có dạng:

```
[B, 1, n_mels, time_steps]
```

Sau CNN, dữ liệu được biến đổi thành chuỗi để đưa vào RNN. Nếu sinh viên sửa model, cần rất cẩn thận với shape. Lỗi shape mismatch là lỗi kinh điển khi làm CRNN.

5.5 Lỗi 5. Nhầm vai trò của CNN và RNN

CNN không thay thế RNN, và RNN cũng không thay thế CNN trong kiến trúc này. CNN học đặc trưng cục bộ trên spectrogram. RNN học quan hệ theo thời gian giữa các đặc trưng đó.

5.6 Lỗi 6. Chỉ nhìn accuracy mà không nhìn loss

Accuracy có thể dao động, đặc biệt với bài toán nhiều lớp và dữ liệu không quá lớn. Nếu chỉ nhìn accuracy mà bỏ qua loss, sinh viên có thể chọn nhầm model. Hãy nhìn đồng thời `val_loss`, `val_acc`, learning curves và confusion matrix.

5.7 Lỗi 7. Kỳ vọng BiLSTM chắc chắn thắng GRU

BiLSTM phức tạp hơn GRU, nhưng không có nghĩa là chắc chắn tốt hơn. Nếu dữ liệu ít, mô hình lớn hơn có thể overfit. Nếu thời gian train tăng nhiều nhưng test accuracy không cải thiện, cần ghi nhận trung thực trong báo cáo.

5.8 Lỗi 8. Không kiểm tra W&B

Nếu quên `--use_wandb`, chưa login, hoặc môi trường không import được wandb, run có thể vẫn chạy nhưng không có dashboard. Bài lab này yêu cầu sinh viên phải nộp bằng chứng đã log thí nghiệm trên W&B.

6 Câu hỏi tự kiểm tra sau lab

Sau khi hoàn thành bài lab, sinh viên cần tự trả lời được các câu hỏi sau:

1. Vì sao Lab 4 dùng log-mel spectrogram làm input chính cho CRNN?
2. CRNN khác 1D-CNN ở Lab 3 ở điểm nào?
3. Trong CRNN, CNN block có nhiệm vụ gì?
4. Trong CRNN, GRU hoặc LSTM có nhiệm vụ gì?
5. Shape `[B, 1, n_mels, time_steps]` có ý nghĩa gì?
6. Vì sao cần dùng fold trong UrbanSound8K thay vì chia ngẫu nhiên tùy tiện?
7. Dấu hiệu overfitting trên learning curves là gì?
8. Confusion matrix cho biết thông tin gì mà accuracy không cho biết?
9. CRNN có chắc chắn tốt hơn 1D-CNN không? Vì sao?
10. Khi nào nên cân nhắc dùng BiLSTM-CRNN?
11. Nếu một run có test accuracy cao hơn nhưng train lâu gấp đôi, có nên chọn không?
12. Trong báo cáo, vì sao cần ghi rõ config của từng run?

Nếu trả lời được các câu hỏi trên bằng chính kết quả thí nghiệm của mình, sinh viên đã đạt mục tiêu quan trọng nhất của bài lab: không chỉ chạy code, mà còn hiểu mô hình đang làm gì và biết cách đánh giá kết quả một cách có căn cứ.